Architectural Standardization of the Stela Platform: Governance, Tasks, and Agentic Orchestration

Executive Summary

The Stela platform stands at a pivotal juncture in its architectural maturation, transitioning from a phase of rapid capability acquisition to one of rigorous operational hardening and standardization. The current ecosystem, defined by the "Stateless" philosophy where context is explicitly loaded rather than implicitly assumed [1], faces a critical challenge: the divergence between its robust constitutional framework (TRUTH.md, AGENTS.md) and its operational execution layer. While the governance artifacts are precise, the "Tasks" that drive daily operations exist as generic legacy artifacts, and the logic governing Agent "Orchestration" is isolated in a static documentation file (docs/library/ORCHESTRATION.md) rather than integrated into the executable core.[1]

This research report provides a comprehensive analysis and execution plan for "Phase Two" (Tasks Subsystem) of the Stela evolution, while laying the precise groundwork for "Phase Three" (Agents) and "Phase Four" (Project Mechanics). The primary objective is to satisfy the operational mandate that **Skills, Tasks, and Agents must be able to work together in any combination**. Achieving this level of composability requires the deconstruction of monolithic artifacts—specifically the migration of data from ORCHESTRATION.md into the modular B-TASK-03 (Agent Orchestration) and the upcoming Agent definitions.

The analysis reveals that the current "Orchestration" model is rigid, locking the system into pre-defined linear workflows (e.g., Plan → Review → Commit).[1] To enable dynamic composability, this logic must be atomized: Role definitions ("Who") must migrate to the docs/library/agents/ directory; Workflow logic ("How") must migrate to B-TASK-03; and Policy constraints ("What") must migrate to S-LEARN skills and B-TASK procedures. This restructuring allows the high-level director, ops/bin/project (Phase Four), to dynamically compose sessions based on immediate context rather than static scripts.

Furthermore, this report addresses the critical "Drift" vectors identified in the repository dump: the lack of a formal Task Registry, the "Context Pollution" caused by a monolithic TASK.md, and the absence of "Stela Canon" standards in the B-TASK files.[1] The proposed solution involves establishing docs/library/INDEX.md as the Single Source of Truth (SSOT) for all operational routines, refactoring the TASK.md interface to act solely as a "Loader," and rewriting B-TASK-01 through B-TASK-06 to include strict Provenance, Scope, and Drift Prevention sections.

The report concludes with the formal generation of **Dispatch Packet DP-OPS-0016**, a concrete execution plan crafted to strictly adhere to the TASK.md canon, ensuring the immediate implementation of these architectural upgrades. This DP serves as the bridge between the current fragmented state and the future "Agentic Equilibrium," where human authority and machine logic interact through precisely defined, auditable contracts.

I. The Stela Constitutional Framework and Operational Ontology

The foundation of the Stela platform is not merely code, but a rigid governance structure that defines the relationship between human intent and machine execution. This structure is codified in TRUTH.md and AGENTS.md, which serve as the immutable constitution of the system. Understanding this framework is prerequisite to analyzing the Tasks and Agents subsystems.

1.1 The Philosophy of Stateless Operations

Stela operates on a "Stateless" paradigm. Unlike traditional systems that rely on persistent memory or implicit knowledge (e.g., "tribal knowledge" held by human developers), Stela assumes that **"Context is not memory; it is a file."** This philosophy dictates that for an agent to be operationally aware, it must be "rehydrated" via a specific set of artifacts listed in ops/lib/manifests/CONTEXT.md.[1] If an artifact is not explicitly loaded, it does not exist for the purpose of the current session.

This approach mitigates "Hallucination" and "Drift" by grounding every action in a verifiable file on disk. The "Open Prompt" (ops/bin/open) and the "State Capture" (ops/bin/dump) are the mechanical implementations of this philosophy, creating a closed loop where the system state is serialized before work begins and verified after work completes.[1] The ops/bin/dump utility is particularly sophisticated in this regard. It supports various formats (chatgpt, claude) and binary handling modes (meta, raw, base64), acknowledging the token limits and input modalities of different Large Language Models (LLMs).[1] By providing a deterministic snapshot of the repository state based on git scope (platform, project, full), the dump binary ensures that the agent's context is exactly aligned with the "Truth" of the repository at a specific commit hash (HEAD).

This "Context Injection" architecture mirrors the emerging industry standard of the **Model Context Protocol (MCP)** and the llms.txt standard.[2] Just as MCP defines how agents access external resources in a structured protocol to prevent fragmentation [2], Stela's ops/bin/dump standardizes the ingestion of the repository state. The llms.txt file in the Stela root [1] serves as the discovery entry point, providing pointers to the canonical documentation and tools, much like the proposed standard for AI crawlers to optimize context.[3] This alignment suggests that

Stela is not just a repository but an "Agent-Ready" environment designed for the next generation of autonomous development.

1.2 The Staffing Protocol and Jurisdictional Logic

The "Staffing Protocol," defined in AGENTS.md, establishes a hierarchy of authority that prevents the AI from overstepping its bounds. This protocol divides labor into three distinct roles, a structure that mirrors the "Agentic Organization" models discussed in recent industry literature regarding human-AI collaboration [5]:

Role	Entity	Jurisdiction	Responsibilities
Operator	Human	Final Authority	Owns decisions, approvals, and secrets. Performs all git commits, pushes, and merges.
Integrator	Lead AI	Governance	Maintains structural integrity, detects drift, and generates Dispatch Packets (DPs).
Contractor	Guest AI	Execution	Executes logic tasks and drafts implementation details within a defined scope.

This separation of concerns is critical for the Tasks Subsystem. The **Integrator** is responsible for selecting the Task from the Registry and defining the Scope in TASK.md. The **Contractor** is responsible for executing the Task and producing the "Receipt." The distinction ensures that the entity defining the work is not the same as the entity performing the work, introducing a necessary layer of auditability.[1]

The "Contractor" role is further constrained by the .github/copilot-instructions.md file [1], which explicitly forbids direct pushes to main and mandates a "Reuse-First" discipline. This ensures that agents do not reinvent existing wheels—a common source of "Code Drift" in AI-generated repositories. This protocol addresses the security concerns highlighted in

research regarding agentic frameworks, where decentralized agents might otherwise execute code with insufficient oversight.[2] By centralizing the "Commit Authority" with the Operator, Stela mitigates the risks associated with autonomous code execution.

1.3 Behavioral Logic and Linguistic Precision

To ensure that the interaction between these roles is precise, AGENTS.md enforces a "Behavioral Logic Standard." A key component of this is "Linguistic Precision," specifically the **"Contraction Prohibition."** Agents are forbidden from using contractions (e.g., "don't," "can't").[1] While this may seem stylistic, it serves a functional purpose: it forces a rhythmic precision that aids in technical auditing and reduces the probability of ambiguity in Large Language Model (LLM) outputs. By forcing the agent to write "do not" instead of "don't," the system subtly enforces a formal, logical stance that correlates with higher-quality code generation.

Furthermore, the standard mandates "Quantitative Reporting" and "Absolute Literalism." Agents must report deviations in exact percentages or file counts, avoiding subjective descriptions like "some files were changed." This rigorous standard must be applied to the refactoring of the B-TASK files in DP-OPS-0016, transforming them from loose checklists into quantitative algorithms. This aligns with the "Chain of Thought" reasoning required for complex agentic tasks, where explicit step-by-step logic reduces the hallucination rate.[6]

II. The Tasks Subsystem (Phase Two Implementation)

The "Tasks Subsystem" represents the operational engine of Stela. A "Task" is defined as the "routine" or execution logic that applies "Skills" (tools) to "Projects" (payloads). The current state of the repository reveals a subsystem in transition, burdened by legacy artifacts and a loose interface.

2.1 Analysis of Current State and Drift

The repository dump [1] indicates the presence of six task files: B-TASK-01 through B-TASK-06. However, the design analysis in Stela Task Standardization.pdf [1] describes these as "generic" artifacts that lack the "Stela Canon" genome. They are likely simple checklists imported from an external context, lacking the specific "Provenance," "Scope," and "Drift Preventer" sections required by the platform's constitution.

The primary vector of "Drift" in the current system is the monolithic nature of TASK.md. Currently, TASK.md functions as both the interface and the definition for some routines. This violates the "Interface/Implementation Separation" principle. Loading thousands of tokens of task definitions into the context window for every session ("Context Pollution") degrades agent performance and increases costs. Furthermore, mixing high-frequency work logs with

low-frequency task definitions creates a "Volatility Mismatch" in the git history.[1]

The tools/dp_lint.sh script [1] currently enforces structure within TASK.md, but it cannot validate the *logic* of the tasks themselves if they are embedded as free text. By moving task definitions to separate files (docs/library/tasks/), we enable independent versioning and validation of the operational logic.

2.2 The Task Registry Design (Interface vs. Implementation)

The solution proposed in the architectural design—and adopted for DP-OPS-0016—is to decouple the definition of work from the active contract of work.

2.2.1 The Registry (docs/library/INDEX.md)

The Registry serves as the Single Source of Truth (SSOT) for all available routines. Instead of a separate TASKS.md manifest (which would create "Manifest Sprawl"), the system leverages the existing docs/library/INDEX.md to map stable identifiers to physical files. This aligns with the "Reuse-First Discipline" mandated by AGENTS.md.[1]

Registry Structure Table:

Topic Key	Title	Path (repo-relative)
task-code-review	B-TASK-01: Code Review Protocol	docs/library/tasks/B-TASK-01.md
task-verification	B-TASK-02: Verification Loop	docs/library/tasks/B-TASK-02.md
task-orchestration	B-TASK-03: Agent Orchestration	docs/library/tasks/B-TASK-03.md
task-planning	B-TASK-04: Implementation Planning	docs/library/tasks/B-TASK-04.md
task-cleanup	B-TASK-05: Dead Code Cleanup	docs/library/tasks/B-TASK-05.md
task-docs-sync	B-TASK-06: Documentation Sync	docs/library/tasks/B-TASK-06.md

The tools/lint_library.sh script [1] already enforces the integrity of this registry, checking for

"Dead Ends" (links to missing files) and "Ghosts" (files existing on disk but not registered). Expanding the registry to include Tasks ensures that no operational logic can exist "off the books."

2.2.2 The Interface (TASK.md)

TASK.md is refactored to act solely as a "Loader." It contains a reference pointer in **Section I (Required Context Load)** that instructs the agent to check the Registry. The "Execution Plan" in a Dispatch Packet (DP) can then simply state "Execute B-TASK-01," relying on the agent to load the specific file on demand. This "Load on Demand" architecture significantly reduces token usage and ensures that agents only access the logic relevant to the immediate objective.

2.3 Standardization of B-TASK Artifacts (The "Stela Canon")

While the initial design document proposed renaming tasks to T-OPS (Operational Tasks), the user has explicitly overridden this requirement, mandating the retention of the B-TASK-XX naming convention [User Query]. This decision prioritizes continuity with existing file structures. However, to satisfy the "Anti-Drift Governance," the internal content of these B-TASK files must be completely rewritten to match the rigorous "Genome" defined for T-OPS in the design document.[1]

Each B-TASK file must now contain the following five sections:

1. **Provenance:** Origin (DP-ID), Jurisdiction, and Revision History.
2. **Scope:** Strictly defined boundaries (e.g., "Production Payloads Only").
3. **Invocation Guidance:** The specific "Trigger" condition for the task.
4. **Drift Preventers:** Hard stop conditions (e.g., "Stop if git status is dirty").
5. **Procedure:** Algorithmic steps formatted as **Action -> Trap -> Solution**.

2.3.1 Detailed Specification of Core Tasks

- **B-TASK-01: Code Review Protocol.** This task elevates the code review from a subjective "look over" to a rigorous security gate. It must integrate with S-LEARN-05 (Security Protocols) and enforce the lint_truth.sh check to catch forbidden terminology. Crucially, it must perform a "Secrets Scan," grepping for patterns like sk-, ghp, and Bearer to prevent credential leaks.[1] This task directly addresses the security gap identified in agentic workflows where permissions are often too broad.[2]
- **B-TASK-02: Verification Loop.** This task replaces simple test commands with a "System Integrity Check." It mandates a clean git state (git status --porcelain) before execution, preventing the "it works on my machine" fallacy. It must also execute tools/verify_tree.sh to ensure that no files have been placed outside the sanctioned docs/ or projects/ directories.[1]
- **B-TASK-04: Implementation Planning.** This task is critical for "Governance-as-Code." It mandates that the "Architect" agent (R-AGENT-01) must not produce conversational

output. Instead, the output must be a valid DP-DRAFT file saved to storage/dp/intake/. This ensures that planning artifacts are durable and can be reviewed by the Operator before execution.[1]

- **B-TASK-05: Dead Code Cleanup.** This task governs hygiene. It requires the use of tools/context_lint.sh to ensure that no "Required Context" files are accidentally deleted. It also enforces a "Registry Check"—if a library file is removed, the Registry (INDEX.md) must be updated in the same transaction.
- **B-TASK-06: Documentation Sync.** This task enforces the "Filing Doctrine": if ops/ (Runs) changes, docs/ops/ (Explains) must be updated. If Canon changes, SoP.md (History) must be updated. This task prevents "Documentation Drift," ensuring the manual matches the machine.[1]

III. Orchestration & Composability (The "Any Combination" Requirement)

The user's query introduces a critical, non-negotiable requirement: **"Skills, Agents, and Tasks must be able to work together in any combinations."** This mandates a Composable Architecture, where the logic governing how these entities interact is not locked into rigid scripts but is modular and dynamically selectable. The current state of docs/library/ORCHESTRATION.md is the primary barrier to this composability, as it conflates role definitions, workflow logic, and policy constraints into a single "floating" artifact.[1]

3.1 Analysis of the Legacy Orchestration Artifact

Based on the repository content [1], docs/library/ORCHESTRATION.md currently contains three distinct types of data:

1. **Agent Definitions:** Descriptions of agents like Planner, Architect, Code-Reviewer, and Security-Reviewer.
2. **Workflow Logic:** Prescriptive sequences like "Feature Implementation Workflow" (Plan → Review → Commit) and "Parallel Task Execution."
3. **Policy Constraints:** Security checklists (e.g., "No hardcoded secrets") and output formats ("Handoff Document Format").

This conflation creates a dependency web that prevents "Any Combination." For example, if the "Feature Workflow" is hardcoded to use the "Planner" agent, an Operator cannot easily substitute a custom "Refactor Agent" without rewriting the orchestration logic.

3.2 The Migration and Decomposition Strategy

To achieve composability, ORCHESTRATION.md must be deconstructed and its atomic components migrated to their proper operational homes. This migration will satisfy the user's

request to move data to B-TASK-03, DP-OPS-0017, and DP-OPS-0018.

3.2.1 Destination A: B-TASK-03 (The Staffing Protocol & Meta-Logic)

The **Workflow Logic** and **Parallel Execution** guidelines must migrate to B-TASK-03. This task becomes the "Operating System" for agent interaction. It defines the *protocol* of handoff, not the specific agents involved.

- **Migration Action:** Extract the "Execution Pattern" and "Handoff Document Format" from ORCHESTRATION.md and embed them into B-TASK-03.[1]
- **Refinement:** Instead of hardcoding "Use Planner Agent," B-TASK-03 will specify: "Integrator invokes the selected Planning Agent (e.g., R-AGENT-01 or R-AGENT-04) via B-TASK-04." This allows the specific agent to be swapped out while the protocol remains constant.
- **Parallelism:** The "Parallel Task Execution" logic [1] moves to B-TASK-03 as a sub-procedure for high-velocity DPs, defining how multiple Contractors can operate on disjoint scopes simultaneously without collision.

3.2.2 Destination B: DP-OPS-0017 (The Agent Registry)

The **Agent Definitions** ("Who") must be migrated to the docs/library/agents/ directory as individual R-AGENT-XX.md files during the Phase Three execution (DP-OPS-0017).

- **Migration Action:**
  - "Planner" becomes R-AGENT-04 (Implementation Planning).
  - "Architect" becomes R-AGENT-01 (System Design).
  - "Code-Reviewer" becomes R-AGENT-02.
  - "Security-Reviewer" becomes R-AGENT-06.
  - "Doc-Updater" becomes R-AGENT-03.
  - "Refactor-Cleaner" becomes R-AGENT-05.
- **Constraint:** These definitions must effectively be removed from ORCHESTRATION.md to prevent "Dual Truths." B-TASK-03 will reference the *Registry* (INDEX.md) rather than the text of the orchestration file.

3.2.3 Destination C: DP-OPS-0018 (The Project Binary)

The **Composability Logic** must inform the design of the ops/bin/project binary in Phase Four.

- **Migration Action:** The concept of "Workflow Types" (feature, bugfix, refactor, security) found in ORCHESTRATION.md [1] should serve as the blueprint for ops/bin/project subcommands.
- **Implementation:** ops/bin/project start feature will programmatically invoke the "Feature Workflow" logic now housed in B-TASK-03, automatically loading R-AGENT-01 and R-AGENT-02. This "hardens" the loose text descriptions into executable binary logic.

3.3 Achieving "Any Combination" through Modularity

By executing this migration, the platform shifts from a "Scripted" architecture to a "Composable" one. The components become atomic building blocks:

- **Tasks (B-TASK):** Define *What* to do (Review, Verify, Clean).
- **Agents (R-AGENT):** Define *Who* does it (Architect, Reviewer).
- **Skills (S-LEARN):** Define *How* to do specific steps (Git Forensics, Security Checks).

Composability Example:

An Operator needs to perform a "Security Audit on Legacy Code."

- **Combination:** R-AGENT-06 (Security Reviewer) + B-TASK-01 (Code Review) + S-LEARN-06 (Git Forensics).
- **Mechanism:** B-TASK-03 (Orchestration) manages the session. It loads R-AGENT-06 as the Contractor. It instructs the Contractor to execute B-TASK-01. When the Contractor encounters a complex git history, it invokes S-LEARN-06 to analyze the "Hot Zone."

This dynamic composition is impossible with the current monolithic ORCHESTRATION.md but becomes the default state once the migration to B-TASK-03 is complete.

IV. The Agents Subsystem (Phase Three Forecast)

Following the implementation of the Tasks Subsystem, the platform moves to Phase Three: The Agents Subsystem (DP-OPS-0017). This phase addresses the redundancy between the AGENTS.md constitution and the docs/library/agents/ directory.

4.1 Refactoring AGENTS.md

Currently, AGENTS.md acts as a hybrid document, containing both high-level constitutional logic and specific role definitions.[1] This violates the "Single Source of Truth" principle if role definitions also exist in docs/library/agents/. The refactoring plan involves stripping AGENTS.md down to its core components:

1. **Staffing Protocol:** The definition of Operator/Integrator/Contractor.
2. **Behavioral Logic:** The standards for Linguistic Precision and Anti-Drift.
3. **Pointer:** A reference to docs/library/INDEX.md for specific specialized agent definitions (R-AGENT-XX).

4.2 The R-AGENT Genome

The R-AGENT files (Roles) must be hardened to operate "just like skills and tasks," as requested by the user. This means they must be "Load on Demand" resources that define how an agent should behave when assuming a specific persona.

Standardized R-AGENT Structure:

- **Jurisdiction:** What files and logic this agent owns.
- **Permissions:** What this agent is allowed to touch (e.g., Architect can touch TASK.md, but Code Reviewer cannot).
- **Trigger:** When this agent should be invoked.
- **Skill Access:** A list of S-LEARN skills this agent is authorized to use.

This formalization aligns with the "System Cards" concept proposed in AI policy literature, where agent behaviors and limitations are explicitly documented for transparency and safety.[7]

V. Project Mechanics (Phase Four Consolidation)

The final phase of this architectural evolution is the consolidation of mechanics via the ops/bin/project binary (DP-OPS-0018). This binary is currently a placeholder (ops/bin/project contains <PLACEHOLDER FILE> [1]). Its implementation will "bring it all together" by serving as the high-level interface for the entire system.

5.1 The ops/bin/project Specification

This binary will serve as the "Director" of the platform, utilizing the logic migrated from ORCHESTRATION.md.

- **project init:** Scaffolds a new project in projects/ using templates from ops/lib/project/templates. It must automatically register the project in ops/lib/manifests/PROJECTS.md.[1]
- **project status:** Reports the current active project, the active TASK.md state, and the last SoP.md entry.
- **project log:** Provides a quick interface to append entries to the project-specific changelog, enforcing the "Work Log" discipline defined in TASK.md.
- **project execute:** The realization of "Any Combination." This command will accept arguments like --agent=architect --task=plan, dynamically loading R-AGENT-01 and B-TASK-04 via the B-TASK-03 orchestration protocol.

5.2 The Unified Theory of Stela Operations

With ops/bin/project operational, the workflow becomes fully "Agentic":

1. **Operator** invokes ops/bin/open to load context.
2. **Operator** invokes ops/bin/project init to create the payload.
3. **Integrator** uses R-AGENT-01 (Architect) to draft a TASK.md (DP) using B-TASK-04.
4. **Contractor** uses R-AGENT-02 (Reviewer) to execute the work, utilizing S-LEARN skills.
5. **System** verifies integrity via B-TASK-02 (Verification).
6. **Operator** closes the session, and ops/bin/project updates the registry.

VI. Implementation Specifications (Dispatch Packet DP-OPS-0016)

The following section presents the concrete implementation artifacts (Dispatch Packet) required to execute this architectural vision. This DP adheres strictly to the TASK.md template [1], the B-TASK naming convention override, and the rigorous standards of Stela Canon.

It explicitly incorporates the migration of ORCHESTRATION.md data into B-TASK-03 to satisfy the "Any Combination" requirement immediately, rather than waiting for future phases. This ensures that the system is "Architecturally Ready" for the Agents and Project Binary phases that follow.

6.1 Critical Implementation Steps

- **Refactor B-TASK-01 to B-TASK-06:** Complete rewrite of the legacy files found in the dump.[1]
- **Migrate Orchestration:** Extract workflows from ORCHESTRATION.md into B-TASK-03.
- **Establish Registry:** Update docs/library/INDEX.md with the new Task section.
- **Harden Template:** Update TASK.md to prevent open loops and enforce the receipt checklist.

This execution plan ensures that Stela transitions from a loose collection of scripts to a hardened, governance-first platform capable of sustaining autonomous agent operations.

DP-OPS-0016: Tasks Subsystem & Registry Implementation

0. FRESHNESS GATE (MUST PASS BEFORE WORK)

Base Branch: work/tasks-sub-system

Required Work Branch: work/tasks-sub-system

Base HEAD: 7e890ab96

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD

- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING ANYTHING)

Worker must confirm loaded before acting:

- OPEN, TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - docs/library/INDEX.md
  - docs/library/tasks/B-TASK-01.md
  - docs/library/tasks/B-TASK-02.md
  - docs/library/tasks/B-TASK-03.md
  - docs/library/tasks/B-TASK-04.md
  - docs/library/tasks/B-TASK-05.md
  - docs/library/tasks/B-TASK-06.md
  - docs/library/ORCHESTRATION.md
  - TASK.md

II) SCOPE & SAFETY

- **Objective:** Implement the Tasks Subsystem by upgrading legacy B-TASK artifacts to Stela Canon standards, migrating orchestration logic to support "Any Combination" composability, establishing the Task Registry, and hardening the TASK.md template.
- **Non-Goals (optional, drift-killer):**
  - No changes to ops/bin/project (Deferred to DP-OPS-0018).
  - No changes to AGENTS.md core constitution (Deferred to DP-OPS-0017).
  - No implementation of new functionality in projects/.

Target Files allowlist (hard gate)

- TASK.md
- SoP.md
- docs/library/INDEX.md
- docs/library/tasks/B-TASK-01.md
- docs/library/tasks/B-TASK-02.md
- docs/library/tasks/B-TASK-03.md
- docs/library/tasks/B-TASK-04.md
- docs/library/tasks/B-TASK-05.md
- docs/library/tasks/B-TASK-06.md
- docs/library/ORCHESTRATION.md (Read-only source for migration, may be modified to deprecate sections)

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **TRUTH.md wins**.
  If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
  - STOP if required DP inputs are missing (do not guess).
  - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

III. EXECUTION PLAN (A–E CANON)

A) STATE

- **Context:** The repository utilizes "generic" B-TASK files that lack Stela Canon rigor. TASK.md acts as a loose template rather than a Registry interface. docs/library/ORCHESTRATION.md contains vital workflow logic (Agent Definitions + Workflow Patterns) that is currently conflated and disconnected from the operational subsystem, preventing modular composability.
- **Drift:**

    - **Unregistered Logic:** B-TASK files exist but are not in INDEX.md.
    - **Legacy Formatting:** Tasks lack "Provenance", "Scope", and "Drift Preventers".
    - **Orchestration Disconnect:** Agent workflows live in a static doc (ORCHESTRATION.md) rather than the executable B-TASK-03, violating the "Any Combination" requirement.
- **Desired State:**
    - **Registry Established:** INDEX.md maps task-keys to files.
    - **Tasks Hardened:** B-TASKs are rewritten with Stela Genome.
    - **Orchestration Migrated:** Workflow logic moved from ORCHESTRATION.md to B-TASK-03 to enable composability.
    - **Interface Hardened:** TASK.md points to the Registry.

B) REQUEST

Numbered tasks (concrete, no vibes):

1. **Refactor B-TASK-01 (Code Review Protocol):**
    - Rewrite docs/library/tasks/B-TASK-01.md.
    - **Provenance:** "System Hardening (DP-OPS-0016)".
    - **Procedure:** Enforce S-LEARN-05 (Security) and S-LEARN-04 (Standards). Include mandatory grep checks for secrets (sk-, ghp, Bearer).
    - **Drift Preventer:** Stop if lint_truth.sh fails.
2. **Refactor B-TASK-02 (Verification Loop):**
    - Rewrite docs/library/tasks/B-TASK-02.md.
    - **Procedure:** Mandate git status --porcelain (Clean State) before execution. Run tools/verify_tree.sh and tools/context_lint.sh.
    - **Output:** Must produce a clean verification receipt.
3. **Refactor B-TASK-03 (Agent Orchestration) & Migrate Logic:**
    - Rewrite docs/library/tasks/B-TASK-03.md.
    - **Source:** Read docs/library/ORCHESTRATION.md.[1]
    - **Migration - Workflow:** Extract "Execution Pattern", "Handoff Document Format", and "Parallel Execution" logic. Embed this into B-TASK-03 as the "Staffing Protocol" (Integrator defines Scope -> Contractor accepts -> Contractor executes).
    - **Migration - Composability:** Define the method for "Any Combination": B-TASK-03 must accept dynamic arguments for Agent and Task selection, rather than enforcing a static "Planner -> Reviewer" loop.
    - **Deprecation:** Add a note to ORCHESTRATION.md (or the new B-TASK-03) stating: "Agent Role Definitions (Who) are migrated to docs/library/agents/ in DP-OPS-0017. Workflow Logic (How) is now defined here in B-TASK-03."
4. **Refactor B-TASK-04 (Implementation Planning):**
    - Rewrite docs/library/tasks/B-TASK-04.md.
    - **Constraint:** Output must be a file storage/dp/intake/DP-DRAFT-XX.md (Governance-as-Code).
    - **Drift Preventer:** Stop if output is conversational text only.

5. **Refactor B-TASK-05 (Dead Code Cleanup):**
    - Rewrite docs/library/tasks/B-TASK-05.md.
    - **Safety:** Mandate tools/context_lint.sh before deletion.
    - **Registry Check:** If library files are deleted, update docs/library/INDEX.md.
6. **Refactor B-TASK-06 (Documentation Sync):**
    - Rewrite docs/library/tasks/B-TASK-06.md.
    - **Procedure:** Enforce Filing Doctrine. If ops/ changes, check docs/ops/. If docs/library/ changes, check INDEX.md.
7. **Update Registry (INDEX.md):**
    - Edit docs/library/INDEX.md.
    - Add a new section 6. Standard Tasks (B-TASK).
    - Register all 6 B-TASK files with stable keys (e.g., task-code-review | Code Review Protocol | docs/library/tasks/B-TASK-01.md).
8. **Harden TASK.md Template:**
    - Edit TASK.md.
    - **Section I:** Add line "See docs/library/INDEX.md for Standard Tasks (B-TASK)."
    - **Section III:** Add comment ``.
    - **Section E (Receipt):** Replace generic bullets with Mandatory Checklist:
        - [ ] storage/handoff/DP-OPS-XX-RESULTS.md created
        - [ ] OPEN artifact preserved
        - [ ] Dump manifest preserved
    - **Allowlist:** Update default allowlist example to include TASK.md (Work Log updates only).

C) CHANGELOG

- **Log (human-readable, 1–6 bullets):**
    - Refactored B-TASK-01 through B-TASK-06 to Stela Canon (Provenance/Drift/Procedure).
    - Migrated Orchestration workflows from ORCHESTRATION.md to B-TASK-03 to enable composability.
    - Registered "Standard Tasks" in docs/library/INDEX.md.
    - Hardened TASK.md to remove Open Loops and enforce Receipt Bundle checklist.
    - Logged DP-OPS-0016 in SoP.md.

D) PATCH / DIFF

- **Format:** Unified diff (preferred) or anchored snippets.

E) RECEIPT (REQUIRED)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent when scope differs)
- If dump refiners are used, include this fallback line in the DP:

    - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- Context lint (use repo-canonical command):
    - Prefer: bash tools/context_lint.sh
    - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when TRUTH/canon surfaces touched; use repo-canonical command):
    - Prefer: bash tools/lint_truth.sh
    - If not found and required: **STOP** and ask (do not invent a substitute)
- Library Guard (New):
    - bash tools/lint_library.sh

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/skill/skill_lib.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution.
  Do not edit the Provenance block.
- Promote: run ops/lib/skill/skill_lib.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include SKILL.md.
- Proof required in RESULTS:
    - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
- Promotion: command invoked plus grep -n "Promotion Packet:" SKILL.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0016-RESULTS.md (required)
- storage/handoff/OPEN-*-*.txt (when produced)
- storage/handoff/OPEN-PORCELAIN-*-*.txt (when produced)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0000-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

3) CLOSEOUT (MANDATORY)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0016-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Ensure proof bundle checklist is satisfied (above).
  (No bundle = DISAPPROVE.)
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones

- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

3.1) THREAD TRANSITION (RESET / ARCHIVE RULE)

Purpose: Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file
   - NEXT: the single action (often "start new DP" or "await operator")
2. Reset sections **1) ACTIVE CONTEXT** and **2) CURRENT DISPATCH PACKET** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:
   - ---
   - THREAD START: DP-OPS-

(If the Work Log gets too long, it's acceptable to keep only the most recent thread's entries here and move older thread logs into the DP RESULTS file—only when explicitly directed.)

---

4) WORK LOG (TIMESTAMPED CONTINUITY)

Rule: This is not a transcript. It's the durable breadcrumb trail.

Expectation: Append a new timestamped entry on every DP closeout (no exceptions).

Each entry ends with: NEXT: <one single action>.

- ** — DP-OPS-0016: Initialized DP to refactor Tasks Subsystem, Registry, and Template. Migrated Orchestration logic for composability. Verification: PENDING. Blockers: none. NEXT: Execute B-TASK refactor.

Works cited

1. dump-full-work-tasks-sub-system-7e890ab96.txt
2. Securing AI Agent Execution - arXiv, accessed February 2, 2026, https://arxiv.org/html/2510.21236v1
3. Working with llms.txt | Platform Overview - Mastercard Developers, accessed February 2, 2026, https://developer.mastercard.com/platform/documentation/agent-toolkit/working-with-llmstxt/
4. Give Your AI Agents Deep Understanding With LLMS.txt | by Dazbo (Darren

Lester) | Google Cloud - Medium, accessed February 2, 2026, https://medium.com/google-cloud/give-your-ai-agents-deep-understanding-with-llms-txt-4f948590332b
5. The agentic organization: A new operating model for AI | McKinsey, accessed February 2, 2026, https://www.mckinsey.com/capabilities/people-and-organizational-performance/our-insights/the-agentic-organization-contours-of-the-next-paradigm-for-the-ai-era
6. AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges - arXiv, accessed February 2, 2026, https://arxiv.org/html/2505.10468v1
7. Understanding Agentic AI: - Information Technology Industry Council (ITI), accessed February 2, 2026, https://www.itic.org/documents/artificial-intelligence/ITI_AgenticAI_Final.pdf